Contrat d'integration des agents

Document de reference, commun aux trois filieres

Version du contrat : 1.0 — applicable aux filieres Cybersecurite, Automatisation metier et IoT.

Ce document decrit precisement comment un agent, quelle que soit l'equipe qui le developpe, doit communiquer avec le noyau pour pouvoir y etre integre. Un agent qui ne le respecte pas sera refuse lors de la verification par l'enseignant, ou suspendu s'il est deja publie.

1. Principe general

Chaque agent est un programme independant, qui expose une API HTTP, et qui tourne separement du noyau (en pratique, dans son propre conteneur Docker). Le noyau ne lit jamais directement le code ou la base de donnees d'un agent : il ne connait de lui que son adresse (base_url) et son manifeste.

- Un agent ne doit jamais appeler directement l'adresse d'un autre agent. Toute collaboration passe par le bus d'echange du noyau (section 6).
- Un agent recoit systematiquement l'identifiant de l'entreprise concernee (X-Tenant-Id) et sa cle d'acces (tenant_api_key) a chaque appel : il ne doit jamais deviner lui-meme quelle entreprise l'appelle.

2. Le fichier manifeste (agent.manifest.json)

Chaque agent fournit, a la racine de son depot, un fichier manifeste conforme a l'exemple suivant (reprenez le fichier agent.manifest.json de l'agent exemple et modifiez-le) :

```
{
  "slug": "mon-agent",
  "name": "Nom de mon agent",
  "version": "1.0.0",
  "description": "Description courte",
  "filiere": "cybersecurite",
  "category": "scan",
  "developer": { "name": "...", "email": "..." },
  "team_name": "Equipe 3",
  "pricing_model": "flat_monthly",
  "price": 0,
  "endpoints": {
    "base_url": "http://mon-agent:8080",
    "health": "/health",
    "activate": "/tenant/activate",
    "deactivate": "/tenant/deactivate",
    "status": "/tenant/{tenant_id}/status",
    "config_schema": "/config-schema",
    "exchange_notify": "/exchange/notify"
  },
  "produces": [],
  "consumes": []
}
```

Le champ filiere accepte exactement trois valeurs : cybersecurite, automatisation_metier, iot. Les champs produces et consumes restent vides par defaut : ils ne sont a remplir que si votre agent echange des donnees avec un autre agent (voir section 6). Un agent qui se contente d'envoyer des alertes n'en a pas besoin.

3. Les 5 endpoints obligatoires

GET /health

```
Reponse : { "status": "ok", "data": { "uptime_seconds": 1234 }, "error": null }
```

POST /tenant/activate

Appele par le noyau quand une entreprise souscrit a l'agent.

```
Requete recue :
  Headers : Authorization: Bearer <tenant_api_key>
            X-Tenant-Id: <company_id>
  Corps   : { "config": { ... } }
Reponse  : { "status": "ok", "data": { "activated": true }, "error": null }
```

POST /tenant/deactivate

Doit supprimer ou anonymiser les donnees du tenant concerne.

```
Reponse : { "status": "ok", "data": { "deactivated": true, "data_purged": true }, "error":
null }
```

GET /tenant/{tenant_id}/status

```
Reponse : { "status": "ok", "data": { "state": "running" }, "error": null }
```

GET /config-schema

Decrit en JSON Schema les parametres que l'entreprise peut renseigner a l'activation.

4. L'enveloppe de reponse standard

Toute reponse HTTP d'un agent, sur tout endpoint, doit respecter cette structure exacte :

```
{ "status": "ok" | "error", "data": { ... } | null, "error": "message" | null }
```

Une reponse qui ne la respecte pas est traitee par le noyau comme une violation du contrat, ce qui suspend automatiquement la souscription concernee.

5. Le webhook d'alertes

Un agent peut signaler un evenement (alerte, notification) au noyau a tout moment :

```
POST /webhooks/agent-events
Headers : Authorization: Bearer <tenant_api_key>
          X-Tenant-Id: <company_id>
Corps   : { "event_type": "...", "severity": "info|low|medium|high|critical", "data": { ... }
}
```

C'est le mecanisme le plus simple, suffisant pour la grande majorite des agents des filieres cybersecurite et IoT : un scanner de vulnerabilites, un capteur de temperature ou un systeme de detection d'intrusion n'ont besoin que de ce canal pour remplir leur role.

6. Le bus d'echange (optionnel, pour les agents qui collaborent)

Certains agents doivent echanger des donnees entre eux, en particulier dans la filiere automatisation metier (un agent Logistique a besoin de connaitre le stock avant de programmer une expedition). Le mecanisme est decrit en detail dans INTER_AGENT_DATA_CONTRACT.docx ; en resume :

- Un agent publie un objet avec POST /exchange/agent/{type}. Il doit avoir declare ce type dans son champ produces.
- Le noyau notifie automatiquement, via POST sur l'endpoint exchange_notify de chaque agent concerne, tout agent actif de la meme entreprise qui declare consommer ce type dans son champ consumes.

- Un humain de l'entreprise peut alimenter ou consulter les memes objets depuis son tableau de bord, meme si elle n'a souscrit a aucun agent correspondant.

Un agent qui n'a pas besoin de ce mecanisme laisse simplement produces et consumes vides dans son manifeste, et n'implemente pas /exchange/notify.

7. Securite et isolation des donnees

- Isolation stricte par tenant : aucune donnee d'une entreprise ne doit etre visible par une autre.
- Aucun secret en dur dans le code (utilisez des variables d'environnement).
- POST /tenant/deactivate doit reellement supprimer les donnees du tenant.
- Aucun appel reseau vers un service non mentionne dans votre rapport d'execution.

8. Packaging

Chaque agent est livre avec un Dockerfile fonctionnel et autonome (voir celui de l'agent exemple). Le port d'ecoute doit correspondre a celui declare dans base_url.

9. Verification avant publication

Avant de certifier un agent, l'enseignant verifie :

- Le manifeste est valide et le slug est unique.
- Les 5 endpoints obligatoires repondent avec l'enveloppe standard.
- L'agent s'active et se desactive correctement depuis la page /admin.
- Si consumes n'est pas vide, /exchange/notify fonctionne avec un objet de test.
- Aucune fuite de donnees entre deux tenants de test.

10. Repartition des responsabilites

	Noyau (enseignant)	Agent (equipe etudiante)
Authentification, entreprises, catalogue	Oui	Non
Bus d'echange (stockage, routage)	Oui	Produit/consomme uniquement
Logique metier de l'agent	Non	Oui
Isolation des donnees du tenant	Verifie	Implemente
Disponibilite de l'agent pendant la demo	Non	Oui